

Project 4

SkillSwap – Peer-to-Peer Skill Exchange Web Application

A Secure Full-Stack Learning Marketplace with Real-Time Communication

1. Introduction and Motivation

SkillSwap is a full-stack peer-to-peer learning platform designed to connect individuals who want to exchange skills directly without monetary transactions. The platform enables users to offer expertise in one domain while learning from others in another domain.

The system was designed not just as a web application, but as a production-ready, secure, real-time platform with structured database design, role management, testing infrastructure, and containerized deployment.

The primary goal was to build a secure, scalable, real-time web platform using modern full-stack technologies while following best engineering practices such as authentication security, middleware protection, testing coverage, and deployment readiness.

2. Problem Statement

Modern learning platforms are often:

- Centralized
- Transaction-based
- Lacking direct peer matching
- Not optimized for real-time interaction

The goal of SkillSwap was to create a decentralized peer-to-peer exchange system where:

1. Users can register securely.
 2. Users can list skills they can teach.
 3. Users can search for skills they want to learn.
 4. Users can match and schedule sessions.
 5. Users can communicate in real-time.
 6. The system ensures authentication and secure API access.
-

3. System Architecture Overview

The application follows a classic three-layer architecture:

Frontend (React 18)

Backend (Node.js + Express.js)

Database (SQLite relational schema)

It also integrates:

- Socket.io for real-time communication
 - JWT-based authentication
 - Security middleware stack
 - Docker containerization
 - Comprehensive test infrastructure
-

4. Frontend Architecture

The frontend was built using:

- React 18
- React Router
- Axios for API communication
- Material UI (MUI) for UI components
- Socket.io client for real-time features

4.1 Component Structure

The frontend is modular and divided into:

- Authentication pages

- Dashboard
- Skill listing interface
- Profile management
- Matching interface
- Real-time chat component
- Notifications system

State management is handled using React hooks and contextual state handling where necessary.

5. Backend Architecture

The backend was built using:

- Node.js
- Express.js

5.1 Core Responsibilities

The backend handles:

- Authentication
- API routing
- Database access
- Skill matching logic
- Session scheduling
- Real-time socket event handling
- Middleware security enforcement

The entry point is `server.js`, which initializes:

- Express app
 - Middleware stack
 - Socket.io server
 - Database connection
 - Route controllers
-

6. Authentication and Security Implementation

Security was a core focus of the project.

6.1 Authentication Mechanism

Authentication was implemented using:

- JSON Web Tokens (JWT)
- bcryptjs for password hashing

Workflow:

1. User registers.
2. Password is hashed using bcrypt.
3. On login, password comparison is performed.
4. A JWT token is generated.
5. Token is attached to protected routes.

JWT ensures:

- Stateless authentication
 - Secure session management
 - Reduced server-side session storage
-

6.2 Middleware Security Stack

The backend uses multiple security layers:

- Helmet → Secures HTTP headers
- CORS → Cross-origin request control
- express-rate-limit → Prevents brute-force attacks
- Input validation checks
- Error handling middleware

This prevents:

- Cross-site scripting attacks
 - Request flooding
 - Malformed payload injection
-

7. Database Design

The system uses SQLite with a structured 9-table relational schema.

7.1 Core Entities

The schema includes:

- Users
- Skills
- Skill categories
- User-skill relationships
- Session bookings
- Chat messages
- Notifications
- Reviews
- Authentication metadata

7.2 Relational Integrity

Foreign keys enforce:

- User ownership of skills
- Session mapping between two users
- Message association to sessions
- Review association to completed sessions

Migration scripts were implemented to:

- Reset database
- Migrate schema
- Seed initial data

This ensures reproducibility and structured deployment.

8. Real-Time Communication System

SkillSwap integrates Socket.io for real-time interaction.

8.1 Real-Time Features

- Live chat between matched users
- Session notifications
- Instant booking updates
- Online presence detection

Socket events include:

- Connection initialization
- Message emission
- Broadcast events
- Notification push

This enables bidirectional communication.

9. Testing Infrastructure

Testing was implemented extensively using:

- Jest
- Supertest
- React Testing Library

9.1 Backend Testing

- Unit tests
- Integration tests
- Acceptance tests

Test cases cover:

- Authentication
 - API endpoints
 - Database queries
 - Error handling
-

9.2 Frontend Testing

- Component rendering
- Interaction testing
- Coverage reporting

Coverage reports include:

- HTML
- LCOV
- Text summaries

This ensures maintainability and reliability.

10. DevOps and Deployment

The project includes:

- Docker containerization
- Separate development and production scripts
- Build scripts for frontend
- Database migration scripts

Docker ensures:

- Environment consistency
 - Deployment reproducibility
 - Cross-system compatibility
-

11. Challenges Encountered

1. Managing real-time socket connections alongside REST APIs.
2. Maintaining relational integrity in SQLite.
3. Preventing JWT misuse.
4. Synchronizing frontend state with real-time backend events.
5. Ensuring test coverage across both frontend and backend.

Solutions involved:

- Middleware layering
 - Controlled state updates
 - Modular route structure
 - Comprehensive testing suite
-

12. Performance Considerations

The backend was optimized to:

- Minimize redundant database queries
- Use batched operations where possible
- Maintain efficient socket communication

SQLite was chosen for lightweight deployment but schema design allows migration to PostgreSQL or MySQL.

13. Skills Demonstrated

- Full-stack development
 - REST API design
 - JWT authentication
 - Password hashing security
 - Real-time system design
 - Relational database modeling
 - Middleware security hardening
 - Testing automation
 - Docker deployment
 - State management in React
-

14. Real-World Applications

This architecture can scale into:

- Peer-to-peer tutoring platforms
 - Mentorship marketplaces
 - Freelancer knowledge exchange systems
 - Internal corporate skill exchange systems
-

15. Future Improvements

- Migration to PostgreSQL
 - WebSocket scaling for distributed systems
 - AI-based skill matching
 - Rating-based recommendation engine
 - Microservices refactor
 - OAuth integration
-

This is now your full detailed documentation for SkillSwap.

Next project will be:

ScandiCart – Full-Stack E-Commerce Marketplace

Say:

NEXT

And I will continue with the same level of depth.